

CHAPTER III. AUTOMATA ĐẦY XUỐNG VÀ NGÔN NGỮ PHI NGỮ CẢNH

§1. VĂN PHẠM PHI NGỮ CẢNH VÀ CÂY SUY DẪN

I. Xuất xứ của văn phạm phi ngữ cảnh

1. Nguồn gốc ra đời

Xuất xứ của văn phạm phi ngữ cảnh (Context-Free Grammar - CFG) là mô tả ngôn ngữ tự nhiên. Văn phạm phi ngữ cảnh được Noam Chomsky giới thiệu năm 1956 – 1959 trong quá trình nghiên cứu cấu trúc hình thức của ngôn ngữ tự nhiên. Mục tiêu của Chomsky là xây dựng các mô hình toán học để mô tả cách con người tạo ra và hiểu câu trong ngôn ngữ tự nhiên, đặc biệt là các cấu trúc lồng nhau (ví dụ: mệnh đề phụ, ngoặc, quan hệ chủ ngữ – vị ngữ).

Một câu sinh ra qua các bước khai triển các cú pháp liên kết từ thành câu. Các nhà ngôn ngữ đặt hy vọng vào mô hình văn phạm phi ngữ cảnh để biểu diễn ngôn ngữ nhưng họ sớm nhận thấy mô hình này quá nhỏ bé, không đủ diễn tả hết sự phong phú và linh hoạt của ngôn ngữ tự nhiên.

Đặc trưng cốt lõi của văn phạm phi ngữ cảnh là mỗi sản xuất của văn phạm đều có dạng $A \rightarrow \alpha$. Do có thể thay thế A bằng α ở bất kỳ đâu, bất kể những ký tự xung quanh A là gì. Nó không phụ thuộc vào "ngữ cảnh" nên văn phạm này được gọi là phi ngữ cảnh.

2. Sự giao thoa với khoa học máy tính

Gần như cùng thời điểm đó, các nhà khoa học máy tính đang đau đầu tìm cách định nghĩa cú pháp cho các ngôn ngữ lập trình đời đầu.

- *John Backus*: Khi thiết kế ngôn ngữ *FORTRAN*, ông đã bắt đầu suy nghĩ về cách mô tả cú pháp.
- *Peter Naur*: Năm 1959, khi soạn thảo báo cáo cho ngôn ngữ *ALGOL 60*, ông đã cải tiến cách ký hiệu của Backus.

Sự thật thú vị: Cách ký hiệu này được gọi là BNF (Backus-Naur Form). Sau đó, người ta mới nhận ra rằng BNF về mặt toán học hoàn toàn tương đương với văn phạm phi ngữ cảnh mà Chomsky đã đề xuất. Dạng chuẩn Backus – Naur được dùng để diễn tả cú pháp của ngôn ngữ lập trình cấp cao chính là một biến thể của văn phạm phi ngữ cảnh, trong đó dấu $:=$ thay thế $\rightarrow$. Một biểu thức đại số có thể định nghĩa:

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ví dụ 1.1.1. Cho biểu thức $(7 - 3) * (3 + 2)$.

a) Cho văn phạm với tập sản xuất trên sinh ra biểu thức trên.

b) Vẽ cây suy dẫn sinh ra biểu thức trên.

c) Vẽ 2 cây suy diễn khác nhau cho biểu thức $5 + 5 * 5$. Từ đó suy ra văn phạm với tập sản xuất trên là văn phạm nhập nhằng.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Do

Việc nghiên cứu văn phạm phi ngữ cảnh tạo nên một cơ sở lý luận cho việc biểu diễn ngôn ngữ lập trình, cho việc tìm kiếm các giải thuật phân tích cú pháp vận dụng trong các chương trình dịch và cho nhiều ứng dụng khác về xử lý các chuỗi.

3. Ý nghĩa và ứng dụng của văn phạm phi ngữ cảnh

Sự xuất hiện của CFG đã thay đổi hoàn toàn cách chúng ta xây dựng phần mềm:

- Trình biên dịch (Compilers): Hầu hết cú pháp của các ngôn ngữ lập trình hiện đại (C, Java, Python) được định nghĩa bằng CFG.
- Trình phân tích cú pháp (Parser): Các công cụ như Yacc, Bison hay Antlr sử dụng CFG để biến mã nguồn thành cây cú pháp (Parse Tree).
- Xử lý ngôn ngữ tự nhiên (NLP): Dùng để phân tích cấu trúc ngữ pháp của câu trước khi đưa vào các mô hình học sâu.

Văn phạm phi ngữ cảnh ra đời từ nỗ lực của **Chomsky** nhằm hiểu ngôn ngữ loài người, nhưng lại trở thành "ngôn ngữ mẹ" để chúng ta giao tiếp với máy tính thông qua các trình biên dịch.

II. Tính chất đóng của văn phạm phi ngữ cảnh

Định lý 1.2.1. Cho L_1, L_2 là hai ngôn ngữ phi ngữ cảnh cùng trên bảng chữ Σ . Khi đó:

- $L_1 \cup L_2, L_1 L_2$ là ngôn ngữ phi ngữ cảnh.
- L_1^R, L_1^* là ngôn ngữ phi ngữ cảnh.

Ví dụ 1.2.2. Cho ngôn ngữ $L = \{0^n 1^n 0^m | m, n \geq 1\}$.

a) L là ghép tiếp của hai ngôn ngữ nào? Xây dựng văn phạm sinh ra hai ngôn ngữ này.

L

Văn phạm

Văn phạm

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) L thuộc lớp ngôn ngữ nào?

G_1

G_2

L

Định lý 1.2.3. Lớp các ngôn ngữ phi ngữ cảnh không đóng với phép giao và phép lấy phần bù.

Ví dụ 1.2.4. Cho ngôn ngữ $L_1 = \{0^n 1^n 2^m | m, n \geq 0\}$, $L_2 = \{0^m 1^n 2^n | m, n \geq 0\}$.

a) Chứng minh L_1, L_2 là các ngôn ngữ phi ngữ cảnh.

Vấn đề

- Nếu

- Nếu

- Nếu

- Nếu

Vậy

Vấn đề

- Nếu

- Nếu

- Nếu

- Nếu

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Vậy

b) $L_1 \cap L_2$ có là ngôn ngữ phi ngữ cảnh không? Giải thích.

Ứng dụng: Tính chất này thường được dùng để chứng minh một ngôn ngữ không phải là phi ngữ cảnh bằng phương pháp phản chứng. Nếu giao một ngôn ngữ "nghi vấn" với một ngôn ngữ chính quy mà kết quả ra một ngôn ngữ không phải ngôn ngữ phi ngữ cảnh, thì ngôn ngữ nghi vấn đó chắc chắn không phải ngôn ngữ phi ngữ cảnh.

Định lý 1.2.5. Nếu L_1 là ngôn ngữ phi ngữ cảnh, L_2 là ngôn ngữ chính quy thì $L_1 \cap L_2$ là ngôn ngữ phi ngữ cảnh.

Ví dụ 1.2.6. Cho ngôn ngữ $L_1 = \{0^n 1^n 2^m | m, n \geq 0\}$, $L_2 = \{0^m 1^n 2^k | m, n, k \geq 0\}$.

a) L_1, L_2 là các ngôn ngữ gì? Giải thích.

L_1 là ngôn ngữ phi ngữ cảnh (theo ví dụ 1.2.4).

L_1

Văn phạm

Văn phạm

Văn phạm

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) $L_1 \cap L_2$ có là ngôn ngữ phi ngữ cảnh không? Giải thích.

Định lý 1.2.7. Nếu thay thế mỗi ký tự trong ngôn ngữ phi ngữ cảnh bằng một ngôn ngữ phi ngữ cảnh khác thì ngôn ngữ nhận được vẫn là ngôn ngữ phi ngữ cảnh.

III. Cây suy dẫn trong văn phạm phi ngữ cảnh

1. Cây suy dẫn

Trong văn phạm phi ngữ cảnh, **cây suy dẫn** (còn gọi là *cây phân tích cú pháp*, *parse tree*) là một biểu diễn dạng cây cho thấy cách một xâu được sinh ra từ ký hiệu bắt đầu thông qua việc áp dụng các luật sinh của văn phạm. Cây suy dẫn phản ánh rõ ràng cấu trúc phân cấp của xâu, vượt trội so với cách biểu diễn tuyến tính bằng suy dẫn.

Định nghĩa 1.3.1.

•

•

•

•

•

•

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định lý 1.3.2. Cho $G = (V, \Sigma, S, P)$ là một văn phạm phi ngữ cảnh và $u \in \Sigma^*$. Khi đó $u \in L(G)$ khi và chỉ khi tồn tại một cây suy dẫn đầy đủ trong G có kết quả là u .

2. Suy dẫn trái và suy dẫn phải

Định nghĩa 1.3.3.

•

•

Ví dụ 1.3.4. Cho văn phạm $G = (\{S, A, B\}, \{0, 1\}, S, \{S \rightarrow 0S \mid AB, A \rightarrow A0 \mid 1B, B \rightarrow B1 \mid 1\})$ và chuỗi $u = 000111000111$.

a) Tìm suy dẫn trái nhất của chuỗi u .

b) Tìm suy dẫn phải nhất của chuỗi u .

c) Vẽ cây suy dẫn của chuỗi u .

IV. Rút gọn văn phạm phi ngữ cảnh

1. Mục đích của việc rút gọn văn phạm phi ngữ cảnh

Trong lý thuyết ngôn ngữ hình thức, một văn phạm phi ngữ cảnh có thể chứa các thành phần không cần thiết nhưng không làm thay đổi ngôn ngữ được sinh ra. Việc rút gọn văn phạm phi ngữ cảnh nhằm loại bỏ các thành phần dư thừa đó, từ đó:

- Làm cho văn phạm đơn giản, gọn gàng và dễ phân tích hơn.
- Thuận lợi cho việc chứng minh lý thuyết.
- Là bước chuẩn bị quan trọng cho các phép chuẩn hóa văn phạm.
- Hỗ trợ xây dựng PDA và trình biên dịch.

Điều quan trọng khi rút gọn văn phạm phi ngữ cảnh là: văn phạm sau khi rút gọn phải sinh ra cùng một ngôn ngữ như văn phạm ban đầu.

2. Ký hiệu vô ích

Định nghĩa 1.4.1.

•

Nhận xét. X là ký hiệu vô ích nếu:

- ❖ Từ một chuỗi chứa X không thể suy dẫn ra bất cứ chuỗi $u \in \Sigma^*$. Ký hiệu X có tính chất này gọi là ký hiệu vô sinh.
- ❖ Từ S không suy dẫn ra bất cứ chuỗi nào chứa X . Ký hiệu X có tính chất này gọi là ký hiệu không đến được.

Ví dụ 1.4.2. Cho văn phạm phi ngữ cảnh $G = (\{S, A, B\}, \{a, b, c\}, S, \{S \rightarrow aSb \mid aA, A \rightarrow bA \mid a, B \rightarrow cB\})$.

➤

➤

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Bổ đề 1.4.3 (Loại bỏ ký hiệu vô sinh). Cho văn phạm phi ngữ cảnh $G = (V, \Sigma, S, P)$ với $L(G) \neq \emptyset$. Khi đó luôn tồn tại văn phạm phi ngữ cảnh $G' = (V, \Sigma', S, P')$ không chứa ký hiệu vô sinh và tương đương với G .

Từ

- Nếu

- Nếu

Lập

Bổ đề 1.4.4 (Loại bỏ ký hiệu không đến được). Cho văn phạm phi ngữ cảnh $G = (V, \Sigma, S, P)$. Khi đó luôn tồn tại văn phạm phi ngữ cảnh $G' = (V', \Sigma', S, P')$ không chứa ký hiệu không đến được và tương đương với G .

Từ

- Đưa

- Nếu

Tiếp tục

Định lý 1.4.5. Mọi ngôn ngữ phi ngữ cảnh khác $\emptyset$ đều có thể sinh từ ngôn ngữ phi ngữ cảnh không chứa ký hiệu vô ích.

Giả sử $L \neq \emptyset$ là ngôn ngữ phi ngữ cảnh được sinh bởi văn phạm phi ngữ cảnh G . Áp dụng lần lượt bổ đề 1.4.3 và 1.4.4 sẽ được văn phạm phi ngữ cảnh G'' tương đương với G sinh ra L và không chứa ký hiệu vô ích.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ví dụ 1.4.6. Cho văn phạm phi ngữ cảnh $G = (\{a, b, c\}, \{S, A, B, C, D, E\}, S, \{S \rightarrow Ab|aC|BD, A \rightarrow bC, B \rightarrow a, C \rightarrow ab, D \rightarrow aD|Db, E \rightarrow ac\})$.

a) Loại bỏ các ký hiệu vô ích trong G .

Bước 1.

-

-

-

-

-

Vậy

Bước 2.

-

-

Vậy

G_2

b) Áp dụng bổ đề 1.4.4 rồi đến bổ đề 1.4.3 cho văn phạm G .

Bước 1.

-

-

Vậy

Bước 2.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

-

-

-

Vậy

Nhận xét. $G_4 \neq G_2$ và trong G_4 vẫn còn ký hiệu vô ích.

Chú ý. Áp dụng bổ đề 1.4.3 rồi mới áp dụng bổ đề 1.4.4.

3. λ – sản xuất

Định nghĩa 1.4.7.

Định nghĩa 1.4.8.

•

•

Bổ đề 1.4.9 (loại bỏ λ – sản xuất). Cho văn phạm phi ngữ cảnh $G = (V, \Sigma, S, P)$. Khi đó tồn tại văn phạm phi ngữ cảnh $G' = (V, \Sigma, S, P')$ không chứa λ – sản xuất và $L(G') = L(G) \setminus \{\lambda\}$.

Xây dựng tập sản xuất P' theo quy tắc sau:

- Nếu

+

+

+

- Lặp

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ví dụ 1.4.10. Cho văn phạm $G = (\{0,1\}, \{S, A, B, C\}, S, \{S \rightarrow A0B|0A|1, A \rightarrow BC|1C0, B \rightarrow C|01, C \rightarrow \lambda\})$.

a) Xây dựng văn phạm phi ngữ cảnh G_1 không chứa λ – sản xuất và $L(G_1) = L(G) \setminus \{\lambda\}$.

Bước 1.

-

-

-

Bước 2.

-

-

-

-

- Giữ

Vậy

b) Xây dựng văn phạm phi ngữ cảnh G_2 không chứa sản xuất đơn và tương đương với G_1 .

Bước 3.

- Đưa

-

Vậy

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

c) Xây dựng văn phạm phi ngữ cảnh G' không chứa ký hiệu vô ích tương đương với G .

Bước 4.

- Do
- Do
- Do

Vậy

Bước 5.

- Đưa
- Do

Vậy

G_4

4. Các sản xuất đơn

Định nghĩa 1.4.11.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Bổ đề 1.4.12

Xây dựng tập sản xuất P' theo quy tắc sau:

- Đưa
- Nếu
- Lặp

Định lý 1.4.13.

Áp dụng lần lượt các bổ đề 1.4.9, 1.4.12, 1.4.3, 1.4.4 xây dựng được văn phạm G' .

Ví dụ 1.4.14. Cho văn phạm phi ngữ cảnh $G = (\{0,1,2\}, \{S, A, B, C, D, E\}, S, \{S \rightarrow A0B|B1|E2, A \rightarrow BD|01, B \rightarrow DD|BD|B1, C \rightarrow C2|12, D \rightarrow \lambda|1, E \rightarrow EE\})$. Xây dựng văn phạm phi ngữ cảnh G' không chứa ký hiệu vô ích, không chứa λ – sản xuất, không chứa sản xuất đơn mà $L(G') = L(G) \setminus \{\lambda\}$.

Bước 1.

-
-

-

Bước 2.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

-

-

-

-

-

-

- Giữ

Vậy

Bước 3.

- Đưa

-

-

-

-

Vậy

Bước 4.

- Do

- Do

- Do

- Do

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Do

Vậy

Bước 5.

- Đưa

- Do

- Do

- Do

Vậy

G_4

§2. HAI DẠNG CHUẨN QUAN TRỌNG

I. Mục đích của việc chuẩn hóa văn phạm phi ngữ cảnh

Chuẩn hóa văn phạm nhằm đưa văn phạm về một dạng tiêu chuẩn với cấu trúc luật sinh đơn giản, thống nhất, nhưng không làm thay đổi ngôn ngữ sinh ra.

Việc đưa văn phạm phi ngữ cảnh về các chuẩn dạng có ý nghĩa đặc biệt trong:

- Chứng minh lý thuyết về ngôn ngữ phi ngữ cảnh.
- Thiết kế và phân tích thuật toán phân tích cú pháp.
- Chứng minh các định lý đóng.
- Xây dựng máy đẩy xuống (PDA) tương đương

Hai chuẩn dạng quan trọng nhất là:

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Chuẩn dạng Chomsky (Chomsky Normal Form – CNF).
- Chuẩn dạng Greibach (Greibach Normal Form – GNF).

II. Dạng chuẩn Chomsky

1. Định nghĩa

Định nghĩa 2.2.1.

Ví dụ 2.2.2.

➤ Văn phạm

➤ Văn phạm

Nhận xét. Văn phạm phi ngữ cảnh G ở dạng chuẩn Chomsky nếu:

- ❖ Không chứa λ – sản xuất.
- ❖ Không chứa sản xuất đơn
- ❖ Các sản xuất mà vế phải chứa cả ký hiệu kết thúc và không kết thúc.
- ❖ Các sản xuất mà vế phải có nhiều hơn hai ký hiệu.

2. Đưa văn phạm về dạng chuẩn Chomsky

Định lý 2.2.3. Cho văn phạm phi ngữ cảnh $G = (V, \Sigma, S, P)$ với $\lambda \notin L(G)$. Khi đó tồn tại văn phạm phi ngữ cảnh $G' = (V, \Sigma', S, P')$ tương đương với G và ở dạng chuẩn Chomsky.

Theo định lý 1.4.13, luôn có thể giả sử G không chứa ký hiệu vô ích, không chứa λ – sản xuất, không chứa sản xuất đơn.

Bước 1.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Bước 2.

- Nếu

- Nếu

Lặp lại

Bước 3.

- Thêm

- Bỏ đi

Sau

Sau

Ví dụ 2.2.4. Đưa văn phạm phi ngữ cảnh $G = (\{0,1\}, \{S,A,B,C\}, S, \{S \rightarrow AB|BC1|101, A \rightarrow 0BC|011, B \rightarrow C1B|0, C \rightarrow 001|10\})$ về dạng chuẩn Chomsky.

Bước 1.

Thêm

- Thay

- Thay

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Thay

- Thay

- Thay

- Thay

- Thay

Vậy

Bước 2.

- Với

- Với

- Với

- Với

- Với

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Với

Vậy

Nhận xét. Trong bước 2, để giảm bớt các ký hiệu không kết thúc thêm vào quá nhiều, tiến hành như sau:

- Với

- Với

- Với

- Với

- Với

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Với

Vậy

Ví dụ 2.2.5. Đưa văn phạm phi ngữ cảnh $G = (\{a, b\}, \{S, A, B\}, S, \{S \rightarrow ABAB | aAB | ABb | bab, A \rightarrow ABa | bba, B \rightarrow bAB | baa\})$ về dạng chuẩn Chomsky.

Bước 1.

Bước 2.

Cách 1. Với

- Với

- Với

- Với

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Với

- Với

- Với

- Với

- Với

- Với

Vậy

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Cách 2. Với

- Với

- Với

- Với

- Với

- Với

- Với

- Với

- Với

- Với

Vậy

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

3. Ý nghĩa và ứng dụng của dạng chuẩn Chomsky

- *Cấu trúc cây nhị phân*: Mọi cây suy dẫn trong CNF đều là cây nhị phân. điều này cực kỳ thuận lợi cho việc lập trình và lưu trữ dữ liệu.
- *Thuật toán CYK*: CNF là điều kiện bắt buộc để chạy thuật toán CYK (Cocke-Younger-Kasami) — một thuật toán dùng để kiểm tra xem một chuỗi có thuộc về một văn phạm hay không với độ phức tạp $O(n^3)$.
- *Giới hạn độ cao của cây*: Nếu chuỗi có độ dài n , cây suy dẫn trong CNF sẽ có đúng $2n - 1$ đỉnh trong.

III. Dạng chuẩn Greibach

1. Định nghĩa

Định nghĩa 2.3.1.

Ví dụ 2.3.2.

Nếu

Nhận xét. Mọi suy dẫn đều bắt đầu bằng ký hiệu kết thúc.

- ❖ Mỗi bước suy dẫn sinh ra ít nhất một ký tự đầu vào.
- ❖ GNF đặc biệt phù hợp với phân tích cú pháp từ trên xuống (top-down parsing).

2. Đưa văn phạm về dạng chuẩn Chomsky

Định lý 2.3.3. Cho văn phạm phi ngữ cảnh $G = (V, \Sigma, S, P)$ với $\lambda \notin L(G)$. Khi đó tồn tại văn phạm phi ngữ cảnh $G' = (V, \Sigma', S, P')$ tương đương với G và ở dạng chuẩn Greibach.

Ví dụ 2.3.4. Cho văn phạm $G = (\{S, A, B\}, \{0, 1\}, S, \{S \rightarrow 0A1B0|01, A \rightarrow 1B0|110, B \rightarrow 01\})$. Đưa văn phạm G về dạng chuẩn Greibach.

- Thêm

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Thay

- Thay

- Thay

Vậy

3. Ý nghĩa và ứng dụng của dạng chuẩn Greibach

- Tính kết thúc: Vì mỗi bước suy dẫn đều tạo ra đúng một ký hiệu kết thúc ở đầu chuỗi, chúng ta luôn biết chắc chắn rằng quá trình sinh chuỗi đang tiến triển và không bị lặp vô tận.
- Máy tự động đẩy xuống (PDA): GNF có mối liên hệ trực tiếp với PDA. Mỗi sản xuất trong GNF tương ứng với một bước chuyển của PDA: đọc một ký tự a và đẩy các biến α vào ngăn xếp.
- Kiểm soát độ dài: Độ dài của suy dẫn cho một chuỗi có độ dài n sẽ đúng bằng n bước.

IV. So sánh CNF và GNF

1. Bảng so sánh CNF và GNF

Đặc điểm	Dạng chuẩn Chomsky (CNF)	Dạng chuẩn Greibach
Quy tắc cơ bản	$A \rightarrow BC$ hoặc $A \rightarrow a$	$A \rightarrow a\alpha$
Ký hiệu bắt đầu	Không bắt buộc bắt đầu bằng ký hiệu kết thúc	Luôn bắt buộc bắt đầu bằng ký hiệu kết thúc
Số ký hiệu vế phải	2 ký hiệu không kết thúc hoặc 1 ký hiệu kết thúc	1 ký hiệu kết thúc với hậu tố là xâu ký hiệu không kết thúc
Có λ – sản xuất	Không (trừ $S \rightarrow \lambda$ nếu cần)	Không
Cây suy dẫn	Cây nhị phân cân đối	Cây lệch về một phía
Thuật toán liên quan	Thuật toán CYK	Automata đẩy xuống (PDA)
Ưu điểm	Dễ tính độ phức tạp	Luôn tạo ra ngay ký hiệu kết thúc
Độ khó chuyển từ CFG	Tương đối dễ (chia nhỏ sản xuất)	Khó hơn (cần khử đệ quy trái)

2. Tại sao cần cả CNF và GNF

Khi xây dựng một trình biên dịch:

- Dùng **CNF** khi muốn kiểm tra xem mã nguồn có đúng cú pháp hay không (nhờ thuật toán CYK rất nhanh).
- Dùng **GNF** khi muốn mô hình hóa cách bộ nhớ (ngăn xếp) hoạt động khi thực thi các câu lệnh lồng nhau.

Cả hai đều giúp loại bỏ sự mơ hồ và những vòng lặp vô tận mà các văn phạm phi ngữ cảnh thông thường hay gặp phải.

§3. AUTOMATA ĐẨY XUỐNG

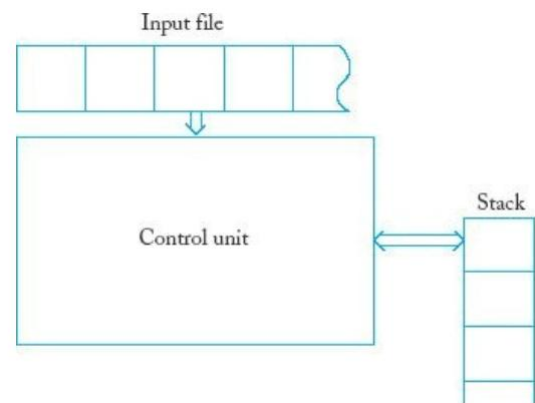
I. Mô tả automata đẩy xuống

Trong chương II ta đã thấy mỗi ngôn ngữ chính quy đều do một văn phạm chính quy sinh ra và đều có một DFA hoặc NFA đoán nhận tức là lớp ngôn ngữ do văn phạm chính quy sinh ra và lớp ngôn ngữ do FA đoán nhận là trùng nhau. Trong chương này ta sẽ thấy lớp ngôn ngữ do các văn phạm phi ngữ cảnh sinh ra trùng với lớp các ngôn ngữ được đoán nhận bởi automata đẩy xuống không đơn định (Nondeterministic Pushdown Automata – NPDA).

Chỉ có các NPDA mới đoán nhận được hết các ngôn ngữ do văn phạm phi ngữ cảnh sinh ra. Lớp các automata đẩy xuống đơn định (Deterministic Pushdown Automata – DPDA) chỉ đoán nhận được một tập con thực sự của lớp các ngôn ngữ phi ngữ cảnh. Tuy nhiên lớp các ngôn ngữ được đoán nhận bởi DPDA khá rộng, bao gồm phần lớn các ngôn ngữ lập trình.

Một PDA cũng giống FA có các thành phần:

- Một băng vào chứa xâu cần đoán nhận.
- Một bộ điều khiển là tập hữu hạn các trạng thái.
- Một đầu đọc cho phép đọc từ trái sang phải các ký hiệu trên băng vào.
- Khác biệt giữa PDA với FA là thêm một băng làm việc, gọi là ngăn xếp (stack), nhờ nó mà khả năng ghi nhớ của PDA được tăng thêm. Ngăn xếp được tổ chức theo nguyên tắc LIFO (Last in first out).



Căn cứ vào trạng thái hiện tại của bộ điều khiển, ký hiệu vào mà đầu đọc quan sát được và ký hiệu ở đầu ngăn xếp, PDA có hai phương án:

-

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

PDA có hai cách đoán nhận xâu như sau:

-

-

Ví dụ 3.1.1. Cho ngôn ngữ $L = \{ucu^R \mid u \in \{a, b\}^*\}$

a) Mô tả ngôn ngữ L .

L là tập các xâu đối xứng trên bảng chữ $\{a, b, c\}$, có ký hiệu c làm tâm đối xứng, trong đó:

L

-

-

b) Xây dựng văn phạm G sinh ra L , từ đó suy ra L thuộc lớp ngôn ngữ nào.

Văn phạm

c) Xây dựng PDA đoán nhận ngôn ngữ L .

PDA

- Ở

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Ở

- Ở

Trường hợp 1.

Trường hợp 2.

-

-

d) Bằng định nghĩa, xây dựng PDA M đoán nhận ngôn ngữ L , từ đó đoán nhận các chuỗi $v_1 = aababcbabaa$ và $v_2 = aababcbabba$.

PDA

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Xâu

Xâu

d) Vẽ đồ thị chuyển mô tả M .

với

Nhận xét. Do ngăn xếp có khả năng lưu giữ các ký hiệu nên PDA có thể ghi nhớ nửa đầu của xâu v rồi sau đó so sánh nó với nửa cuối của xâu v . FA không có khả năng ghi nhớ này.

II. Automata đẩy xuống không đơn định

Định nghĩa 3.2.1 Một automata đẩy xuống không đơn định (NPDA) là một bộ bảy $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ trong đó:

- Q là tập hữu hạn các trạng thái bên trong của bộ điều khiển.
- Σ là tập hữu hạn khác $\emptyset$ các ký hiệu, gọi là bảng chữ vào. Mỗi ký hiệu thuộc Σ gọi là ký hiệu vào.
- Γ là tập hữu hạn khác $\emptyset$ các ký hiệu, gọi là bảng chữ ngăn xếp.
- $q_0 \in Q$ là trạng thái đầu.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- $z_0 \in \Gamma$ là ký hiệu đặc biệt, gọi là ký hiệu bắt đầu ngăn xếp.
- $F \subset Q$ gọi là tập các trạng thái kết thúc.
- $\delta: Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow$ một tập con hữu hạn của $Q \times \Delta^*$, được gọi là hàm chuyển trạng thái của M .

Định nghĩa 3.2.2.

Khi đó

Định nghĩa 3.2.3.

Khi đó

Ở mỗi thời điểm, tình huống tức thời của NPDA M được xác định bởi ba yếu tố sau:

- Xâu

- Trạng

- Phân

Định nghĩa 3.2.4.

Giả sử

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Nếu

- Nếu

Tương tự

Ví dụ 3.2.5. Cho PDA $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, b, z_0\}, \delta, q_0, z_0, \{q_2\})$ với hàm chuyển trạng thái δ xác định như sau: $\delta(q_0, a, z_0) = \{(q_0, a)\}$, $\delta(q_0, a, a) = \{(q_0, aa)\}$, $\delta(q_0, \lambda, a) = \{(q_1, a)\}$, $\delta(q_1, b, a) = \{(q_1, \lambda)\}$, $\delta(q_0, \lambda, z_0) = \delta(q_1, \lambda, \lambda) = \{(q_2, \lambda)\}$.

a) Vẽ đồ thị chuyển mô tả M .

b) Cho M đoán nhận chuỗi a^3b^3 , từ đó chứng minh $L(M) = \{a^n b^n | n \geq 0\}$.

Xâu

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Xâu

Nếu

Nếu

Dãy

c) Xây dựng DPDA M_1 cũng đoán nhận ngôn ngữ $L(M)$.

Trong

Xây

Nếu

Nếu

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Dãy

III. Văn phạm phi ngữ cảnh và automata đẩy xuống

1. Ngôn ngữ được đoán nhận bởi automata đẩy xuống

Định nghĩa 3.3.1.

•

•

•

Ký hiệu

$T(M)$ được gọi là ngôn ngữ được đoán nhận bởi M theo tập trạng thái kết thúc.

Định nghĩa 3.3.2.

•

•

•

Ký hiệu

$N(M)$ được gọi là ngôn ngữ được đoán nhận bởi M theo ngăn xếp rỗng.

Ý nghĩa: Khi thiết kế PDA, ta có quyền chọn cách nào thuận tiện nhất cho bài toán của mình. Thông thường, khi chuyển đổi từ văn phạm phi ngữ cảnh (CFG) sang PDA, cách dùng ngăn xếp rỗng thường đơn giản và trực quan hơn.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

2. Mối liên hệ giữa văn phạm phi ngữ cảnh và automata đẩy xuống

Định lý 3.3.3. Cho ngôn ngữ phi ngữ cảnh L . Khi đó tồn tại một NPDA M đoán nhận L theo tập trạng thái kết thúc.

Giả sử

-

-

-

Hàm chuyển trạng thái δ được xây dựng như sau:

-

-

-

-

Khi đó

Chú ý.

- ✓ Khi ký hiệu ở đỉnh ngăn xếp là ký hiệu không kết thúc thì đầu đọc sẽ không đọc, PDA sẽ thực hiện bước chuyển nhầm mắt.
- ✓ Khi ký hiệu đầu đọc được và ký hiệu ở đỉnh ngăn xếp trùng nhau thì đầu đọc dịch sang phải một ô, giữ nguyên trạng thái q_1 , ký hiệu ở đỉnh ngăn xếp bị đẩy ra và không có ký hiệu được đẩy vào.

Ví dụ 3.3.4. Cho văn phạm phi ngữ cảnh $G = (\{a, b, c\}, \{S, A, B\}, S, \{S \rightarrow aSb \mid bAa, A \rightarrow bAa \mid aBb, B \rightarrow aBb \mid acb\})$.

a) Tìm $L(G)$.

Vậy

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) Xây dựng PDA M đoán nhận $L(G)$ theo tập trạng thái kết thúc và cho M đoán nhận chuỗi $u = aabaaacbbbabb$.

- Do

- Do

- Do

Vậy

Xâu

Định lý 3.3.5. Cho PDA M . Khi đó tồn tại một NPDA M' sao cho $N(M') = T(M)$.

Giả sử $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$. Xây dựng $M' = (Q', \Sigma', \Gamma', \delta', q'_0, z'_0, F')$ như sau:

- $Q' = Q \cup \{p_1, p_2\}$ với $p_1, p_2 \notin Q$.
- $q'_0 = p_1$.
- $\Gamma' = \Gamma \cup \{\$, \lambda\}$, $\$ \notin \Gamma$.
- $z_0 = \$$.
- $F' = \emptyset$.

Hàm chuyển trạng thái δ xác định như sau:

- $\delta'(p_1, \epsilon, \$) = \{(q_0, \$z_0)\}$.
- $\delta'(q, a, z) = \delta(q, a, z) \quad \forall q \in Q, a \in \Sigma, z \in \Gamma$.
- $\delta'(q, \lambda, z) = \begin{cases} \delta(q, \lambda, z) & \text{khi } q \in Q \setminus F, z \in \Gamma \\ \delta(q, \lambda, z) \cup \{(p_2, \lambda)\} & \text{khi } q \in F, z \in \Gamma \end{cases}$.
- $\delta'(q, \lambda, \$) = \{(p_2, \lambda)\}$ nếu $q \in F$.
- $\delta'(p_2, \lambda, z) = \{(p_2, \lambda)\}$.

Khi đó có thể chứng minh được rằng $N(M') = T(M)$.

Định lý 3.3.6. Cho PDA M . Khi đó tồn tại một văn phạm phi ngữ cảnh G sao cho $L(G) = N(M)$.

Nhận xét.

- ❖ Theo định lý 3.3.3, lớp ngôn ngữ phi ngữ cảnh nằm trong lớp ngôn ngữ được đoán nhận bởi NPDA theo tập trạng thái kết thúc.
- ❖ Theo định lý 3.3.5, lớp ngôn ngữ được đoán nhận bởi PDA theo tập trạng thái kết thúc nằm trong lớp ngôn ngữ được đoán nhận bởi NPDA theo ngăn xếp rỗng.
- ❖ Theo định lý 3.3.6, lớp ngôn ngữ được đoán nhận bởi PDA theo tập ngăn xếp rỗng nằm trong lớp ngôn ngữ được sinh bởi văn phạm phi ngữ cảnh.

Định lý 3.3.7. Một ngôn ngữ là phi ngữ cảnh khi và chỉ khi nó được một NPDA đoán nhận.

IV. Automata đẩy xuống đơn định và ngôn ngữ phi ngữ cảnh đơn định

Định nghĩa 3.4.1. Một automata đẩy xuống $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ được gọi là đơn định (DPDA) nếu nó được định nghĩa như trong định nghĩa 3.2.1 với các hạn chế sau:

- $\delta(q, a, z)$ chứa nhiều nhất một phần tử $\forall q \in Q, a \in \Sigma \cup \{\lambda\}, z \in \Gamma$.
- Nếu $\delta(q, \lambda, z) \neq \emptyset$ với $q \in Q, z \in \Gamma$ thì $\delta(q, b, z) = \emptyset \quad \forall b \in \Sigma$.

Điều kiện đầu tiên yêu cầu đối với bất cứ trạng thái nào, đầu đọc đang đọc ký hiệu gì và giá trị của ngăn xếp là bao nhiêu thì chỉ có tối đa một bước chuyển. Điều kiện thứ hai là nếu có thể có

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

bước chuyển “nhắm mắt” ở trạng thái q , ký hiệu ngăn xếp z thì không thể có bước chuyển thông thường.

Định nghĩa 3.4.2. *Ngôn ngữ L được gọi là ngôn ngữ phi ngữ cảnh đơn định khi và chỉ khi tồn tại một DPDA M thỏa mãn $L = L(M)$.*

Chú ý.

- ✓ Trong FA: DFA và NFA có sức mạnh ngang nhau (đều đoán nhận ngôn ngữ chính quy).
- ✓ Trong PDA: NPDA mạnh hơn DPDA.
- NPDA: Đoán nhận toàn bộ lớp ngôn ngữ phi ngữ cảnh (CFL).
- DPDA: Chỉ đoán nhận một tập con gọi là ngôn ngữ phi ngữ cảnh đơn định (DCFL).

V. Văn phạm phi ngữ cảnh mô tả định dạng dữ liệu JSON và liên hệ CFG – PDA – JSON Parser

1. Giới thiệu chung

JSON (JavaScript Object Notation) là một định dạng dữ liệu dạng văn bản, nhẹ và độc lập nền tảng, được sử dụng rộng rãi trong các hệ thống phần mềm hiện đại như dịch vụ web (REST API), ứng dụng web, hệ thống phân tán và cơ sở dữ liệu *NoSQL*. JSON cho phép biểu diễn dữ liệu có cấu trúc phân cấp thông qua các đối tượng (object) và mảng (array), với khả năng lồng nhau ở mức bất kỳ.

Do mang đặc trưng cấu trúc lồng nhau và quan hệ thứ bậc, cú pháp của JSON không thể được mô tả đầy đủ bằng văn phạm chính quy, mà cần sử dụng văn phạm phi ngữ cảnh. Đây là một ví dụ thực tế tiêu biểu cho vai trò của văn phạm phi ngữ cảnh trong khoa học máy tính hiện đại, bên cạnh các ngôn ngữ lập trình.

2. Các thành phần cú pháp của JSON

Về mặt cú pháp, một tài liệu JSON được xây dựng từ các thành phần cơ bản sau:

- Giá trị (value):

- Đối tượng (object):

- Mảng (array):

- Chuỗi (string):

Mỗi tài liệu JSON hợp lệ luôn tương ứng với đúng một cây cú pháp, phản ánh cấu trúc phân cấp của dữ liệu.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

3. Văn phạm phi ngữ cảnh mô tả JSON (dạng chuẩn hóa)

Bảng chữ

Ký hiệu

Object :

Members :

Pair :

Value :

Array :

Elements :

Vì sao đây là CFG?

- Có đệ quy lồng nhau (*object* trong *object*, *array* trong *array*)
- Không cần thông tin ngữ cảnh
- Mỗi sản xuất có dạng $A \rightarrow \alpha$.

JSON không phải ngôn ngữ chính quy, vì:

- Không thể kiểm tra ngoặc $\{$ và $]$ lồng nhau bằng FA.
- Cần ngăn xếp nên là PDA.

4. Tại sao JSON cần văn phạm phi ngữ cảnh?

Việc sử dụng CFG cho JSON không chỉ là lý thuyết mà mang lại những lợi ích thực tế sau:

- Xử lý cấu trúc lồng nhau vô hạn: Giống như cách máy PDA có thể dùng ngăn xếp để kiểm tra các chuỗi nhập dài và phức tạp như aabaaacbbabb, các trình phân tích JSON (JSON Parsers) sử dụng ngăn xếp để theo dõi các lớp ngoặc $\{$ hoặc $[$ đang mở.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Phân tích cú pháp (Parsing): Các quy tắc trên cho phép trình biên dịch xây dựng một Cây phân tích cú pháp (Parse Tree). Cây này giúp máy tính hiểu được "địa chỉ" của từng mẫu dữ liệu trong một tệp JSON không lỗi.
- Kiểm tra tính hợp lệ: Nếu một tệp JSON thiếu một dấu ngoặc đóng hoặc thừa một dấu phẩy, trình phân tích cú pháp dựa trên CFG sẽ phát hiện ra ngay lập tức vì chuỗi đó không thể được sinh ra từ bộ quy tắc.

Bất cứ ai ngừng học tập đều
già đi, dù ở tuổi 20 hay 80. Bất
cứ ai tiếp tục học tập đều trẻ
mãi!

HENRY FORD